



INSTITUTO FEDERAL
Rio Grande do Sul



Conceitos de Orientação a Objetos

Orientação a Objetos

- A orientação a objetos (O.O) pressupõe que o mundo é composto por objetos
- A modelagem (projeto) de sistemas O.O exige um certo grau de **abstração**
- **Os conceitos discutidos aqui são importantes para o entendimento do Diagrama de Classes**

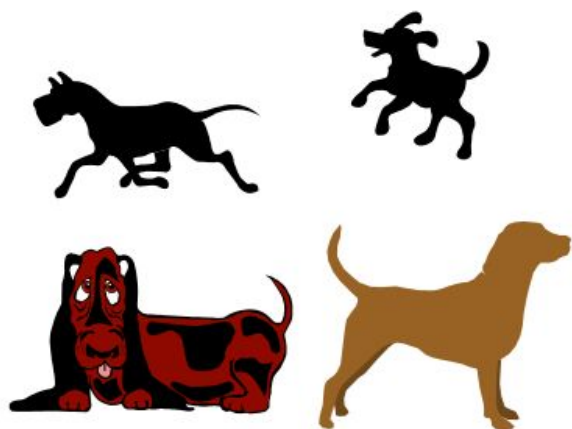
Conceitos

- Objeto e classe
- Classificação
- Abstração
- Instanciação
- Representação gráfica de classes
 - Atributos e métodos
 - Associação (Simples, Agregação e Composição)
 - Herança
 - Polimorfismo
 - Sobrecarga

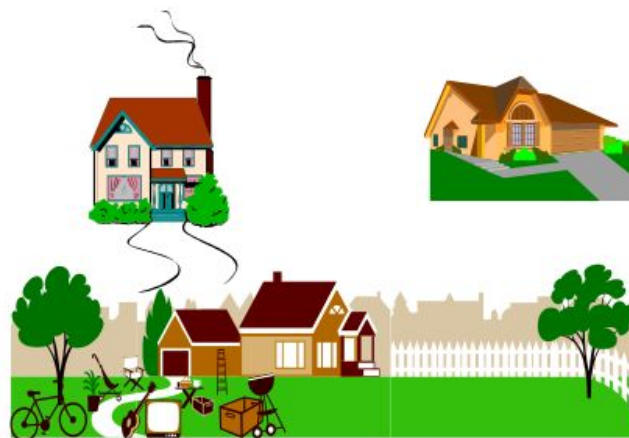
Objeto e Classe

- **Classe** é uma estrutura que abstrai um conjunto de objetos com características similares
 - Descreve os serviços providos por seus objetos (**métodos**) e quais informações eles podem armazenar (**atributos**)
- **Objeto** é uma entidade que tem uma *identidade*
 - Podem representar entidades concretas (carro, bicicleta, pessoa, ...) ou entidades conceituais (estratégia de jogo, ...)
 - Cada **objeto ter sua identidade** significa que dois objetos são distintos mesmo que eles apresentem exatamente as mesmas característica

Objeto e Classe



CACHORRO



CASA



INSTITUTO FEDERAL
Rio Grande do Sul

Objeto e Classe



Classificação, abstração e instanciação

- O ser humano, no início de sua infância, aprende e pensa de uma maneira orientada a objetos, representando seu conhecimento por meio de **classificação e abstrações**
- Uma criança aprende conceitos simples como pessoa, carro e casa e ao fazer isso definem classes
 - Qualquer coisa que possua cabeça tronco e membros passa a ser uma pessoa, qualquer construção onde as pessoas possam entrar passa a ser uma casa
 - Esse processo já exige um grande esforço de **abstração**, pelas diferenças que os objetos de cada classe podem apresentar. Porém, mesmo diferentes, possuem a mesma classificação

Classificação

- Definir os objetos que pertencem à mesma classe
- Exemplo
 - Classe carro:
 - 4 rodas, portas, luzes, farol, freio, vidros
 - Transportam pessoas



Abstração

- Enxergar o **CONCEITO**, chegando à conclusão que um termo (classe) se refere a muitos objetos que possuem **ALGUMAS** características semelhantes
- Após abstrair um conceito inicial, costumamos criar sub-divisões (subclasses) dentro das classes
 - Dentro da classe carro, podemos criar um sub-grupo, classificando-os por marca, modelo, etc

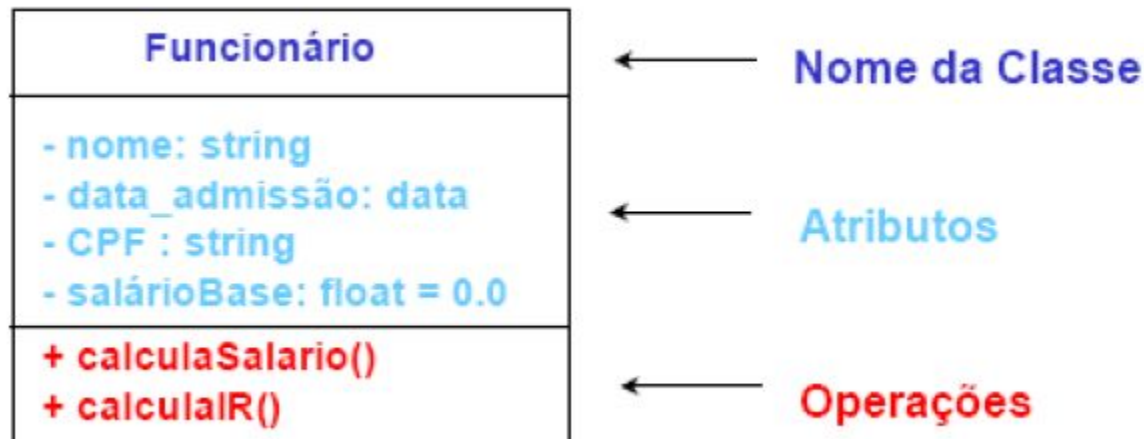


Instanciação

- Quando instanciamos um objeto de classe, estamos criando um novo item do conjunto representado por essa classe

Representação gráfica de Classes

- É representada por um retângulo que pode possuir até 3 divisões

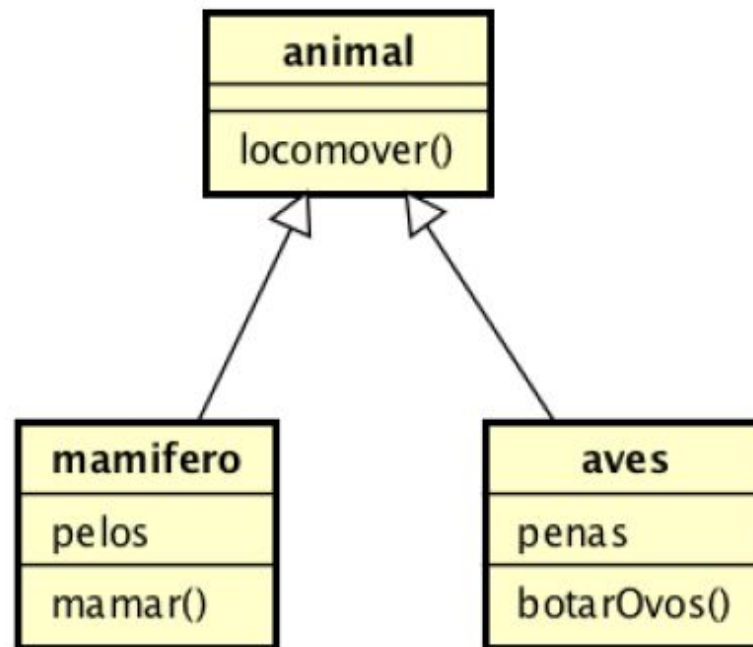


- Normalmente, as classes apresentam atributos e métodos, mas é possível encontrar classes que contenham **apenas uma** destas características

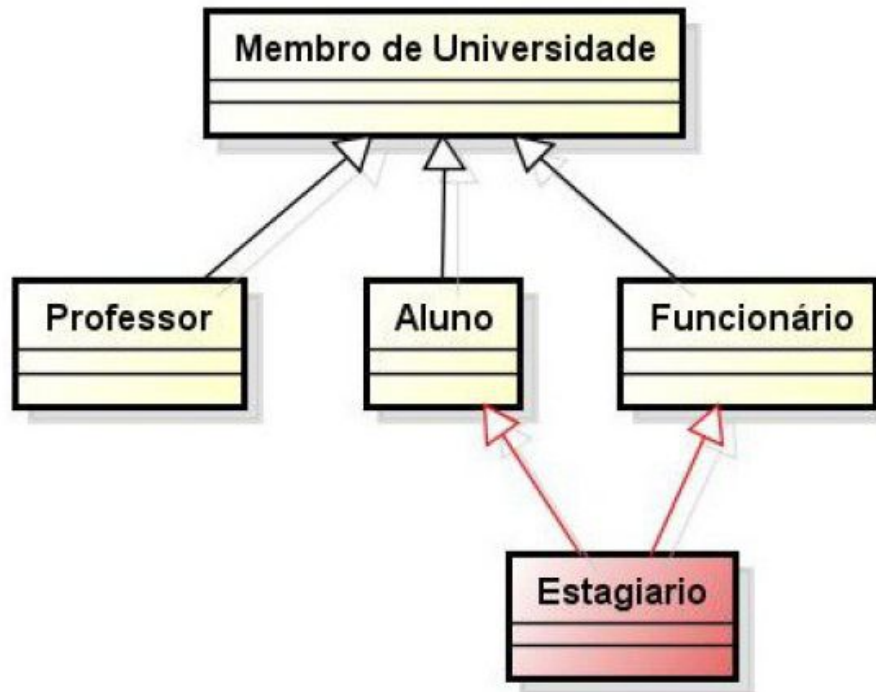
Herança

- Uma das características mais poderosas da O.O., permite o reaproveitamento de atributos e métodos, otimizando o tempo de desenvolvimento, além de permitir a diminuição de linhas de código, bem como facilitar futuras manutenções
- Trabalha com o conceito de **superclasse** e **sub-classe**
 - **Superclasses (classe pai):** classe que possui outras classes derivadas dela
 - **Sub-classe (classe filha):** ao serem derivadas de uma superclasse herdam seus atributos e métodos

Herança



Herança Múltipla



Java → Não suporta

Suporta Herança Múltipla:

- C++
- Python

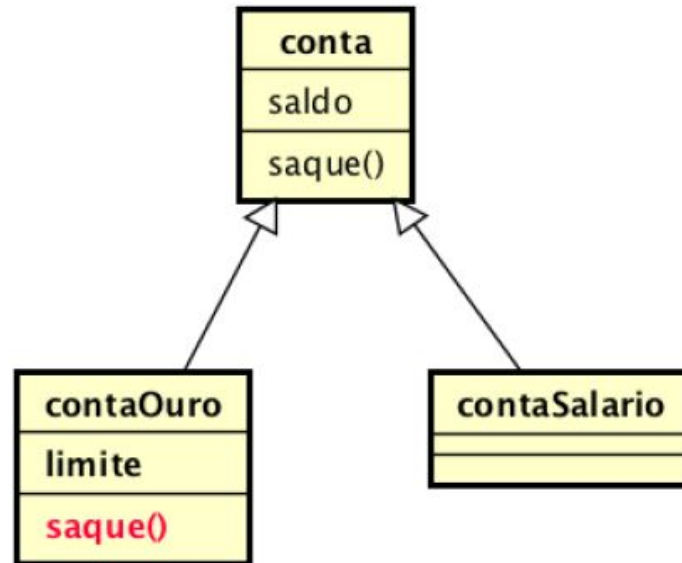
Polimorfismo e Sobrecarga



Polimorfismo

- Permite que classes mais abstratas representem o comportamento das classes concretas
- **Forma mais comum:**
 - Trabalha com a redeclaração de métodos herdados de uma superclasse
 - Embora semelhantes, esses métodos diferem na forma de implementação, necessitando ser reimplementado
 - Dessa maneira podem existir dois ou mais métodos com a mesma nomenclatura

Polimorfismo



Sobrecarga

- Permite a existência de vários métodos de mesmo nome, porém com assinaturas levemente diferentes
- Variando no número, tipo de argumentos, no valor de retorno e até variáveis diferentes

Soma
<pre>+soma(x:int,y:int): int +soma(x:string,y:string): string +soma(x:double,y:double): double</pre>